

University of Asia Pacific
Department of Computer Science and Engineering
REPORT: 03

Course Code: CSE 402

Course Title: Operating System Lab

Submitted by:

Name: Rajash Majumdar

Student ID:23101144

Semester : 4th Year 1st Semester

Section: C2

Submitted to:

Atia Rahman Orthi,
Lecturer
University of Asia Pacific

Problem Statement:

Implement FCFS and SJF CPU Scheduling for the following scenario where the number of processes = 5 and the process information is:

P id	AT	BT
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

Calculate CT, TAT, WT, AVG TAT and AVG WT for both FCFS and SJF scheduling algorithms. Also represent the process execution sequence and compare the average waiting time of FCFS and SJF.

Why is FCFS required in CPU Scheduling?

FCFS is one of the simplest CPU scheduling algorithms. It executes processes according to their arrival order, meaning the process that arrives first gets the CPU first.

FCFS is important because it is:

- **Simple and easy to implement**

- Easy to understand and manage

- Fair in terms of arrival order

- Useful as a basic method for understanding CPU scheduling

However, FCFS does not consider the burst time of a process.

Therefore, a long process arriving earlier can make shorter processes wait for a long time.

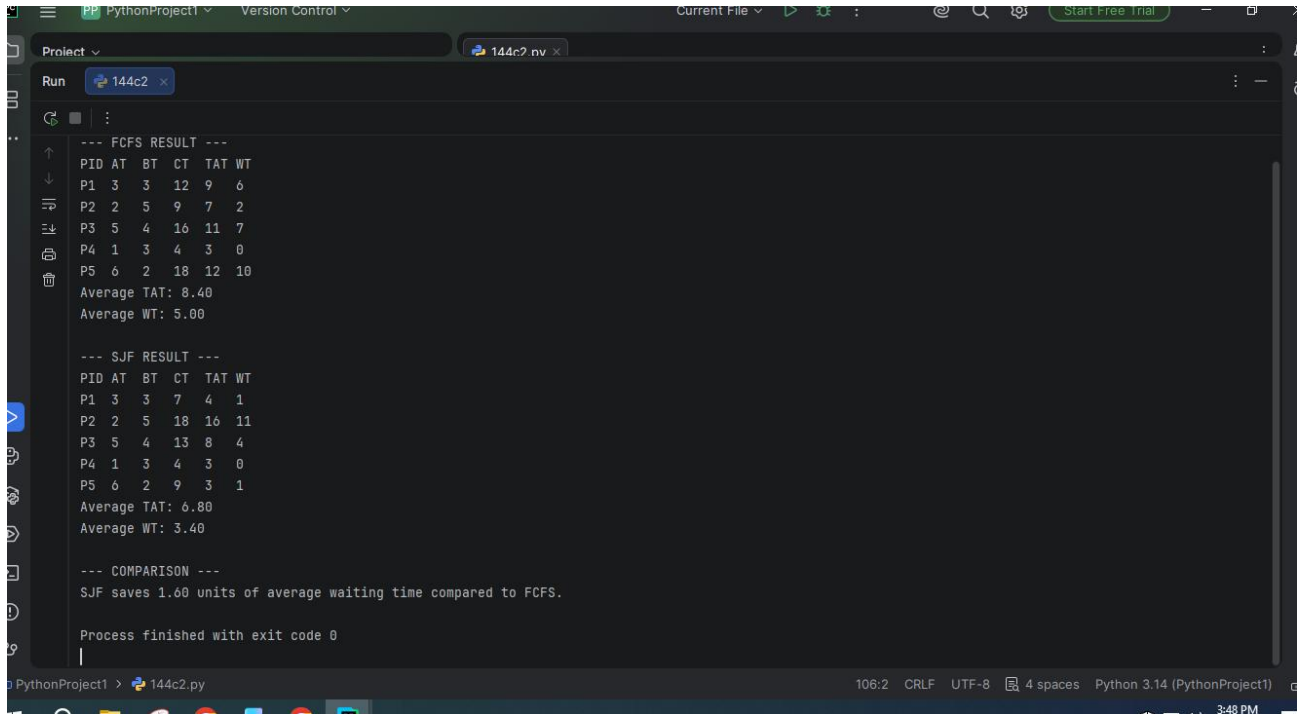
When Does SJF Perform Better Than FCFS?

SJF performs better than FCFS when there are shorter processes waiting behind longer processes. Instead of simply following arrival order, SJF selects the available process with the shortest burst time.

For example, if a long process is executed first in FCFS and several short processes are waiting, the short processes may experience high waiting times. SJF can execute those shorter processes earlier and therefore reduce the average waiting time and turnaround time.

In this experiment, the SJF algorithm can be compared with FCFS using the calculated average waiting time to see which algorithm performs better for the given process set.

Output of the code:



```
--- FCFS RESULT ---
PID AT  BT  CT  TAT WT
P1  3   3   12  9   6
P2  2   5   9   7   2
P3  5   4   16  11  7
P4  1   3   4   3   0
P5  6   2   18  12  10
Average TAT: 8.40
Average WT: 5.00

--- SJF RESULT ---
PID AT  BT  CT  TAT WT
P1  3   3   7   4   1
P2  2   5   18  16  11
P3  5   4   13   8   4
P4  1   3   4   3   0
P5  6   2   9   3   1
Average TAT: 6.80
Average WT: 3.40

--- COMPARISON ---
SJF saves 1.60 units of average waiting time compared to FCFS.

Process finished with exit code 0
```

The screenshot shows a terminal window with the output of a Python script. The script compares the First-Come-First-Served (FCFS) and Shortest-Job-First (SJF) scheduling algorithms. The output displays the results for five processes (P1 to P5) and the average Turnaround Time (TAT) and Waiting Time (WT) for each algorithm. The comparison shows that SJF saves 1.60 units of average waiting time compared to FCFS.

Git repository link:

<https://github.com/Rajash144/LAB-3-SJF-Comparison-with-FCFS>

Discussion:

In this experiment, both FCFS and SJF were implemented using Python. FCFS selects processes based on their arrival time, while SJF selects the available process with the shortest burst time.

While solving the problem, a common challenge is correctly handling processes that have not yet arrived. Another challenge is keeping track of the current CPU time and calculating CT, TAT, and WT correctly.

The results show that SJF can reduce average waiting time compared to FCFS when shorter jobs are available, but SJF requires more decision-making than FCFS. FCFS is simpler, while SJF can provide better scheduling performance in suitable situations.

Conclusion:

The experiment successfully implemented and compared FCFS and SJF CPU scheduling algorithms. FCFS is simple and follows the arrival order, while SJF prioritizes shorter processes. The comparison shows that SJF can achieve a lower average waiting time than FCFS when the process pattern is suitable. Therefore, FCFS is preferred for simplicity, whereas SJF can be preferred when reducing average waiting time is more important.